SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 – PART I

Name: A. Abhishek Reddy

Rg no:192424192

Course Code: CSA0601

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

TASK 1: ALGORITHM DESIGN REVIEW

Diagnose and Repair Poor Pseudocode

Introduction

A logistics company wants to load packages into a delivery vehicle. Each package contains a particular weight and value. The vehicle has a limited carrying capacity. Packages can be divided into smaller portions if necessary. The main objective is to maximise the total value of the packages without exceeding the vehicle capacity.

This problem is known as the Fractional Knapsack Problem. A greedy algorithm is suitable because packages can be selected according to their value per unit of weight.

1.1 Diagnose the Design (2 Marks)

The given pseudocode is incomplete because it does not clearly explain how packages are selected, how much weight is loaded and how the total value is calculated.

Algorithmic deficiencies

S.No.	Deficiency	Explanation
1	Missing input details	It does not specify package weight, package value and vehicle capacity.
2	Unclear arrangement	It does not explain how packages should be arranged or sorted.
3	Undefined selection rule	The meaning of the best package is not clearly explained.
4	Missing feasibility condition	It does not check whether the package fits within the remaining capacity.
5	Fractional selection missing	It does not explain how to select a portion of a package.

6	Remaining capacity not updated	It does not subtract the loaded weight from the available capacity.
7	Unclear termination	It does not specify when the loop should stop.
8	Incomplete result calculation	It does not explain how the total value is calculated.

Explanation:

These deficiencies make the algorithm difficult to implement because different programmers may interpret the selection process differently. A complete algorithm must clearly define the inputs, selection criteria, feasibility conditions and output.

1.2 Repair the Design (3 Marks)

Greedy strategy:

Calculate the value per unit weight of every package. Select the package with the highest value per unit weight first. If the complete package cannot fit, select only the required fraction.

Formula:

[Density[i]=\frac{Value[i]}{Weight[i]}]

Corrected pseudocode

ALGORITHM Fractional Knapsack (Weight, Value, n, Capacity)

INPUT:

Weight[1..n]

Value[1..n]

Number of packages n

Vehicle Capacity

OUTPUT:

Selected fractions

Maximum Total Value

BEGIN

TotalValue ← 0

Remaining ← Capacity

FOR i ← 1 TO n DO

Fraction[i] ← 0

Density[i] ← Value[i] / Weight[i]

END FOR

```
    SORT packages in descending order of Density
    FOR each package i DO
        IF Remaining = 0 THEN
            BREAK
        END IF
        IF Weight[i] ≤ Remaining THEN
            Fraction[i] ← 1
            Remaining ← Remaining - Weight[i]
            TotalValue ← TotalValue + Value[i]
        ELSE
            Fraction[i] ← Remaining / Weight[i]
            TotalValue ← TotalValue +
                        Fraction[i] × Value[i]
            Remaining ← 0
        END IF
    END FOR
    RETURN Fraction, TotalValue
END
```

Explanation of the algorithm:

1. The algorithm reads the weight and value of every package.
2. It calculates the value density of each package.
3. Packages are sorted in descending order of value density.
4. It selects the complete package if enough capacity is available.
5. Otherwise, it selects a fraction of the package to fill the remaining capacity.
6. It updates the total value and remaining capacity.
7. Finally, it returns the selected fractions and maximum total value.

1.3 Design Defence (1 Mark)

The corrected solution is pseudocode because it explains the algorithm using simple, language-independent instructions. It contains inputs, outputs, conditions, loops, calculations and return statements. It does not follow the syntax of any particular programming language. It clearly explains the greedy selection rule and allows another programmer to implement the algorithm in any suitable language.

TASK 2: FROM BUSINESS REQUIREMENT TO GREEDY PSEUDOCODE

Job Sequencing with Deadlines

Introduction

Job Sequencing with Deadlines is a greedy optimization problem. A cloud-service company receives multiple computational jobs. Each job requires one processing slot and has a deadline and profit. A job earns its profit only when it is completed on or before its deadline.

The objective is to schedule jobs in such a way that the total profit is maximised.

2.1 Model the Requirement

Input:

- Job ID
- Deadline of each job
- Profit of each job
- Number of available jobs

Output:

- Final job schedule
- Selected jobs
- Maximum total profit

Objective:

To maximise the total profit earned by completing jobs within their deadlines.

Constraints:

1. Each job requires exactly one processing slot.
2. Only one job can be processed in one slot.
3. Every selected job must finish before or on its deadline.
4. A job cannot be scheduled more than once.
5. Jobs that cannot be scheduled within their deadlines must be rejected.

Decision to be made:

The algorithm must decide which jobs should be selected and in which available slots they should be placed to obtain maximum profit.

Information required:

The algorithm needs the job deadline, profit, available slots and current schedule to determine whether a job can be scheduled.

2.2 Formulate a Greedy Hypothesis

Greedy-choice rule:

Sort the jobs in descending order of profit and schedule each job in the latest available slot before or on its deadline.

Justification:

Selecting higher-profit jobs first helps maximise total profit. Placing each job in the latest available slot preserves earlier slots for jobs having smaller deadlines. This strategy produces an optimal schedule for the given unit-time independent job sequencing problem.

2.3 Design the Algorithm

Pseudocode

ALGORITHM Job Sequencing (Jobs, n)

INPUT:

Jobs with ID, Deadline and Profit

Number of jobs n

OUTPUT:

Final Schedule

Maximum Total Profit

```
BEGIN
  Find maximum deadline D
  FOR i ← 1 TO D DO
    Schedule[i] ← EMPTY
  END FOR
  SORT Jobs in descending order of Profit
  Total Profit ← 0
  FOR each job J DO
    FOR slot ← J.Deadline DOWNTO 1 DO
      IF Schedule[slot] = EMPTY THEN
        Schedule[slot] ← J.ID
        Total Profit ← Total Profit + J.Profit
        BREAK
      END IF
```

```
        END FOR
    END FOR
    RETURN Schedule, Total Profit
END
```

Explanation:

1. Find the maximum deadline to determine the number of available slots.
2. Initialise all slots as empty.
3. Sort the jobs according to descending profit.
4. Consider each job from highest profit to lowest profit.
5. Search for an empty slot starting from its deadline and moving backwards.
6. Place the job in the latest available slot.
7. Add its profit to the total profit.
8. Return the final schedule and total profit.

2.4 Desk-Check (1 Mark)

The given input is:

Job	Deadline	Profit
A	2	80
B	1	35
C	2	60
D	3	40
E	1	25

Step 1: Sort jobs according to profit.

Order	Job	Deadline	Profit
1	**A**	**2**	**80**
2	**C**	**2**	**60**
3	**D**	**3**	**40**
4	**B**	**1**	**35**
5	**E**	**1**	**25**

Maximum deadline = 3.

Therefore, three slots are available.

Step 2: Schedule the jobs.

Job	Available slot checked	Decision	Total Profit
A	Slot 2	Place A in slot 2	80
C	Slot 2 occupied; slot 1 empty	Place C in slot 1	140
D	Slot 3	Place D in slot 3	180
B	Slot 1 occupied	Reject B	180
E	Slot 1 occupied	Reject E	180

Final schedule

Slot	Selected Job	Profit
1	C	60
2	A	80
3	D	40
	Total	180

Final Answer:

Selected jobs = C, A, D.

Maximum total profit:

[60+80+40=boxed{180}]

2.5 Challenge Your Design (1 Mark)

Modified input:

Change only the profit of job B from 35 to 90.

Job	Deadline	Original Profit	Modified Profit
A	2	80	80
B	1	35	90
C	2	60	60
D	3	40	40

E	1	25	25

Prediction:

The schedule will change because job B now has the highest profit. It will be selected first and placed in slot 1.

Modified decision trace

Job	Decision	Slot	Total Profit
B	Selected	1	90
A	Selected	2	170
C	Rejected	—	170
D	Selected	3	210
E	Rejected	—	210

Modified final schedule

Slot	Job	Profit
1	B	90
2	A	80
3	D	40
	Total	210

Conclusion:

The prediction is correct. Changing the profit of job B changes the greedy ordering and the first scheduling decision. The new maximum profit is 210 instead of 180. This demonstrates that the greedy algorithm makes decisions according to the profit of each job while checking deadline feasibility.

TASK 3 – FROM RECURRENCE TO EXECUTABLE LOGIC

Dynamic Programming 0/1 Knapsack

Introduction

The 0/1 Knapsack problem is an optimization problem in which a vehicle has limited carrying capacity. Every item has a particular weight and value. The objective is to select items that provide the maximum total value without exceeding the vehicle capacity.

Items cannot be divided. Each item must either be completely selected or completely rejected.

Dynamic programming is used because it stores the results of smaller problems and uses them to solve larger problems.

3.1 Define the State

The dynamic programming state is:

[DP[i][w]]

Definition:

DP[i][w] represents the maximum total value that can be obtained by considering the first i items with a knapsack capacity of w, without exceeding that capacity.

Where:

- i represents the number of items considered.
- w represents the available capacity.
- DP[i][w] represents the maximum value obtained.

Example:

DP[2][5] represents the maximum value that can be obtained using the first two items with a capacity of 5.

3.2 Develop the Recurrence

There are two possible decisions for every item:

1. Exclude the current item

If the item is not selected, the value remains:

[DP[i][w]=DP[i-1][w]]

This means the current item is rejected and the algorithm uses the best result from the previous items.

2. Include the current item

If the weight of the item is less than or equal to the available capacity, it can be selected.

[Value[i]+DP[i-1][w-Weight[i]]]

This calculates the value of the selected item together with the best value obtainable using the remaining capacity and previous items.

Final recurrence relation

[DP[i][w]= begin{cases} DP[i-1][w], & Weight[i]>w max(DP[i-1][w], Value[i]+DP[i-1][w-Weight[i]]), & Weight[i]\leq w \end{cases}]

Boundary conditions

[DP[0][w]=0]

[DP[i][0]=0]

Explanation:

- When there are no items, the maximum value is zero.
- When the capacity is zero, no item can be selected.
- When an item is too heavy, it must be rejected.
- Otherwise, the algorithm compares the include and exclude values and selects the greater one.

3.3 Translate the Recurrence into Pseudocode

Algorithm

ALGORITHM Knapsack01(Weight, Value, n, W)

INPUT:

Weight[1..n]

Value[1..n]

Capacity W

OUTPUT:

Maximum Value

Selected Items

```
BEGIN
  CREATE DP[0..n][0..W]
  FOR w ← 0 TO W DO
    DP[0][w] ← 0
  END FOR
  FOR i ← 0 TO n DO
    DP[i][0] ← 0
  END FOR
  FOR i ← 1 TO n DO
    FOR w ← 1 TO W DO
      IF Weight[i] > w THEN
        DP[i][w] ← DP[i-1][w]
      ELSE
        Include ← Value[i] +
```

```
                DP[i-1][w-Weight[i]]
            Exclude ← DP[i-1][w]
            DP[i][w] ← MAX(Include, Exclude)
        END IF
    END FOR
END FOR
Maximum Value ← DP[n][W]
Selected Items ← EMPTY
w ← W
FOR i ← n DOWNTO 1 DO
    IF DP[i][w] ≠ DP[i-1][w] THEN
        ADD item i TO Selected Items
        w ← w - Weight[i]
    END IF
END FOR
RETURN Maximum Value, Selected Items
END
```

Explanation:

1. Create a two-dimensional DP table.
2. Initialise the first row and first column with zero.
3. Consider each item one by one.
4. Check whether the current item fits into the available capacity.
5. Calculate the include and exclude values.
6. Store the maximum value in the DP table.
7. After completing the table, obtain the maximum value from DP[n][W].
8. Trace backwards through the table to identify the selected items.
9. Return the maximum value and selected item IDs.

3.4 Desk-Check

The given items are:

Item	Weight	Value

P	2	12
Q	3	20
R	4	25
S	5	30

Vehicle capacity = 7.

Dynamic Programming Table

Items considered	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
No items	0	0	0	0	0	0	0	0
P	0	0	12	12	12	12	12	12
P, Q	0	0	12	20	20	32	32	32
P, Q, R	0	0	12	20	25	32	37	45
P, Q, R, S	0	0	12	20	25	32	37	45

Step-by-step calculation

Step 1: Select item P.

Weight = 2, Value = 12.

The value at capacity 2 becomes 12.

Step 2: Consider item Q.

Weight = 3, Value = 20.

At capacity 5, P and Q can be selected together.

Total weight = 2 + 3 = 5.

Total value = 12 + 20 = 32.

Step 3: Consider item R.

Weight = 4, Value = 25.

At capacity 7, Q and R can be selected together.

Total weight = 3 + 4 = 7.

Total value = 20 + 25 = 45.

Step 4: Consider item S.

Weight = 5, Value = 30.

At capacity 7, P and S provide 42, which is less than 45. Therefore, the previous best value remains unchanged.

Final result

Selected Item	Weight	Value
Q	3	20
R	4	25
Total	7	45

[Maximum Value= boxed{45}]

Selected items = Q and R.

3.5 Explain the Dependency

Dynamic programming states depend on previously calculated states. To calculate DP[i][w], the algorithm requires the previous row DP[i−1][w]. When an item is included, it also requires DP[i−1][w−Weight[i]].

Therefore, these previous states must be calculated before the current state.

The algorithm processes the DP table row by row, starting from the first item and moving towards the last item. This ensures that all required values are available.

If the states are calculated in the wrong order, the algorithm may use incomplete or incorrect values. Therefore, processing in dependency order is important for obtaining the correct maximum value.

Complexity analysis

Type	Complexity	Explanation
Time complexity	O(nW)	Each item is considered for every capacity.
Space complexity	O(nW)	A two-dimensional DP table is maintained.

TASK 4 – PSEUDOCODE FORENSICS

Find, Explain and Repair an Algorithmic Defect

Introduction

The Word Break problem determines whether a given text can be divided completely into valid words from a dictionary.

For example:

Text = "applepie"

Dictionary = pie

The text can be divided into two valid words: "apple" and "pie". Therefore, the result is TRUE.

Dynamic programming is used to check whether every part of the text can be segmented correctly.

4.1 Locate the Defect

The given pseudocode contains a logical defect in the inner IF condition.

Faulty condition

```
IF substring(Text, j, i) is in Dictionary THEN
   DP[i] ← TRUE
   BREAK
END IF
```

Explanation of the defect:

The algorithm checks only whether a substring exists in the dictionary. It does not check whether the characters before that substring can also be divided into valid words.

Finding a valid word at the end of a prefix does not guarantee that the complete prefix can be segmented.

The algorithm must check two conditions:

1. The previous prefix must already be segment table.
2. The current substring must exist in the dictionary.

Correct decision condition

```
IF DP[j] = TRUE AND
  substring(Text, j, i) is in Dictionary THEN
   DP[i] ← TRUE
   BREAK
END IF
```

4.2 Construct a Failing Input (2 Marks)

Test input

Text = "ab"

Dictionary = b

The correct answer should be FALSE because the character "a" is not present in the dictionary.

Trace of the faulty algorithm

Initially:

DP[0] = TRUE

Position i	Substring checked	Dictionary match	DP[i]
1	a	No	FALSE
2	ab	No	FALSE
2	b	Yes	TRUE

At position 2, the algorithm finds "b" in the dictionary and sets DP[2] = TRUE.

However, DP[1] = FALSE. This means the previous character "a" cannot be segmented.

Therefore, DP[2] becomes the first incorrect state.

Comparison

Result	Output
Faulty algorithm	TRUE
Correct algorithm	FALSE
Reason	Character "a" cannot be segmented

The faulty algorithm produces an incorrect result because it does not verify the previous prefix.

4.3 Repair the Algorithm

Only the required IF condition is changed. The remaining structure of the algorithm is retained.

Corrected pseudocode

ALGORITHM Word Break(Text, Dictionary)

INPUT:

Text

Dictionary

OUTPUT:

TRUE or FALSE

```
BEGIN
  n ← LENGTH(Text)
  DP[0] ← TRUE
  FOR i ← 1 TO n DO
    DP[i] ← FALSE
```

```
    FOR j ← 0 TO i-1 DO
      IF DP[j] = TRUE AND
        substring(Text, j, i) is in Dictionary THEN
        DP[i] ← TRUE
        BREAK
      END IF
    END FOR
  END FOR
  RETURN DP[n]
END
```

Explanation:

The corrected algorithm checks whether the previous prefix is valid before accepting the current dictionary word. This prevents invalid prefixes from being marked as TRUE.

4.4 Regression Check (1 Mark)

A regression test checks whether the corrected algorithm continues to work for an input that was already handled correctly.

Additional test input

Text = "applepie"

Dictionary = pie

Trace

Position	Substring	Previous DP state	Result
0	Empty prefix	TRUE	TRUE
5	apple	DP[0] = TRUE	TRUE
8	pie	DP[5] = TRUE	TRUE

The text is successfully divided into:

apple + pie

Therefore:

[DP[8]=TRUE]

Final result: TRUE.

Conclusion:

The corrected algorithm produces the correct result for both valid and invalid inputs. Adding the DP[j] condition repairs the logical defect without changing the complete algorithm.

TASK 5 – SAME GRAPH, DIFFERENT ALGORITHMIC REASONING

Dijkstra, Bellman-Ford and Floyd (7 Marks)

Introduction

Graph algorithms are used to find the shortest paths between vertices in a weighted graph. Different algorithms are suitable for different graph conditions.

- Dijkstra's algorithm uses a greedy strategy.
- Bellman-Ford handles negative-weight edges.
- Floyd's algorithm finds shortest paths between every pair of vertices.

The selection of an algorithm depends on the edge weights and the required output.

Given Directed Weighted Graph

Edge	Weight
A → B	4
A → C	7
B → C	2
B → D	6
C → D	1
D → E	3
C → E	8

5.1 Greedy Shortest-Path Design – Dijkstra

Objective:

Find the shortest distance from source vertex A to all other vertices.

Algorithm steps:

1. Initialise the distance of every vertex as infinity.
2. Set the source vertex distance to zero.
3. Select the unvisited vertex with the smallest distance.
4. Check all outgoing edges from that vertex.
5. Update the distance if a shorter path is found.

6. Update the predecessor of the vertex.
7. Repeat until all reachable vertices are processed.

Pseudocode

```
ALGORITHM Dijkstra(Graph, Source)
INPUT:
   Weighted Graph
   Source Vertex A
OUTPUT:
   Shortest Distances
   Predecessors
BEGIN
   FOR each vertex v DO
      Distance[v] ← INFINITY
      Predecessor[v] ← NIL
      Visited[v] ← FALSE
   END FOR
   Distance[A] ← 0
   WHILE an unvisited reachable vertex exists DO
      u ← Unvisited vertex with minimum Distance
      Visited[u] ← TRUE
      FOR each neighbour v of u DO
         IF Visited[v] = FALSE AND
            Distance[u] + Weight(u,v) < Distance[v] THEN
            Distance[v] ← Distance[u] + Weight(u,v)
            Predecessor[v] ← u
         END IF
      END FOR
   END WHILE
   RETURN Distance, Predecessor
END
```

Desk-check of Dijkstra's algorithm

Initial distances:

Vertex	Distance from A
A	0
B	∞
C	∞
D	∞
E	∞

Step 1: Select A

From A:

- A → B = 4
- A → C = 7

Updated distances:

Vertex	Distance
A	0
B	4
C	7
D	∞
E	∞

Step 2: Select B

B has the smallest unvisited distance of 4.

From B:

- B → C = 4 + 2 = 6
- B → D = 4 + 6 = 10

The distance of C changes from 7 to 6.

Vertex	Distance
A	0

B	4
C	6
D	10
E	∞

Step 3: Select C

C has distance 6.

From C:

- C → D = 6 + 1 = 7
- C → E = 6 + 8 = 14

The distance of D changes from 10 to 7.

Vertex	**Distance**
A	0
B	4
C	6
D	7
E	14

Step 4: Select D

From D:

- D → E = 7 + 3 = 10

The distance of E changes from 14 to 10.

Vertex	**Distance**
A	0
B	4
C	6
D	7
E	10

Final shortest distances

Destination	Shortest Path	Distance
A	A	0
B	A → B	4
C	A → B → C	6
D	A → B → C → D	7
E	A → B → C → D → E	10

Final Answer: The shortest distance from A to E is 10.

5.2 Change the Graph Assumption – Bellman-Ford

Problem:

The graph must now support negative-weight edges.

Dijkstra's algorithm assumes that all edge weights are non-negative. When negative edges are introduced, a vertex that was previously considered final may later receive a shorter path.

Therefore, the greedy selection decision can become unsafe.

Bellman-Ford algorithm

Bellman-Ford repeatedly relaxes all edges. It can handle negative-weight edges and also detects reachable negative-weight cycles.

Pseudocode

```
ALGORITHM BellmanFord(Graph, Source)
BEGI
   FOR each vertex v DO
      Distance[v] ← INFINITY
      Predecessor[v] ← NIL
   END FOR
   Distance[Source] ← 0
   FOR i ← 1 TO |V|-1 DO
      FOR each edge (u,v) DO
         IF Distance[u] ≠ INFINITY AND
            Distance[u] + Weight(u,v) < Distance[v] THEN
            Distance[v] ← Distance[u] + Weight(u,v)
            Predecessor[v] ← u
```

```
        END IF
      END FOR
    END FOR
    FOR each edge (u,v) DO
      IF Distance[u] ≠ INFINITY AND
        Distance[u] + Weight(u,v) < Distance[v] THEN
        RETURN "Negative Weight Cycle Detected"
      END IF
    END FOR
    RETURN Distance, Predecessor
END
```

Explanation:

1. Initialise all distances to infinity.
2. Set the source distance to zero.
3. Relax every edge repeatedly for $|V|-1$ passes.
4. Update the distance whenever a shorter path is found.
5. Perform one additional pass to check for a reachable negative-weight cycle.
6. If a distance can still be reduced, a negative-weight cycle exists.

Why greedy selection is not sufficient:

A negative edge may reduce the distance to a vertex after that vertex has already been selected. Bellman-Ford avoids permanent greedy decisions by repeatedly relaxing all edges.

5.3 Change the Requirement – Floyd's Algorithm

Objective:

Find the shortest paths between every pair of vertices in the graph.

State definition

[D(k)[i][j]]

It represents the shortest distance from vertex i to vertex j when only vertices 1 to k are allowed as intermediate vertices.

Base condition:

- Distance from a vertex to itself is zero.
- Directly connected vertices have their edge weights.

- If no direct edge exists, the distance is infinity.

State transition

[D(k)[i][j]= \min left(D(k-1)[i][j], D(k-1)[i][k]+D(k-1)[k][j] right)]

Pseudocode

```
ALGORITHM Floyd(Graph)
BEGIN
   Initialise Distance matrix D
   FOR each vertex k DO
      FOR each vertex i DO
         FOR each vertex j DO
            IF D[i][k] ≠ INFINITY AND
              D[k][j] ≠ INFINITY THEN
              D[i][j] ← MIN(
                 D[i][j],
                 D[i][k] + D[k][j]
              )
            END IF
         END FOR
      END FOR
   END FOR
   RETURN D
END
```

Explanation:

1. Create a distance matrix for all vertices.
2. Initialise the matrix using the given edge weights.
3. Consider every vertex as an intermediate vertex.
4. Check whether travelling through the intermediate vertex gives a shorter path.
5. Update the distance matrix.
6. Return the shortest distances between every pair of vertices.

Comparison of the three algorithms

Feature	Dijkstra	Bellman-Ford	Floyd
Strategy	Greedy	Repeated relaxation	Dynamic programming
Purpose	Single-source shortest path	Single-source shortest path	All-pairs shortest path
Negative edges	Not supported	Supported	Supported
Negative cycle detection	No	Yes	Can detect through negative diagonal entries
Main operation	Select minimum distance	Relax all edges	Update distance matrix

Conclusion:

Dijkstra is suitable when all edge weights are non-negative. Bellman-Ford is used when negative edges are present. Floyd's algorithm is useful when shortest distances are required between every pair of vertices.

TASK 6 – DESIGN CHALLENGE

Energy-Aware Job Scheduling in a Cloud Data Centre (13 Marks)

Introduction

A cloud data centre operates an energy-efficient computing server. Several computational jobs are waiting to be processed during a maintenance period.

Each job contains a start time, finish time and energy-saving value. The server can execute only one job at a time. Therefore, two jobs that overlap cannot both be selected.

The main objective is to select a set of non-overlapping jobs that produces the maximum total energy-saving value.

This problem is known as Weighted Interval Scheduling. Dynamic programming is suitable because the algorithm must compare different combinations of compatible jobs to find the maximum value.

Stage 1 – Interpret the Problem

Input:

- Job ID
- Start time
- Finish time
- Energy-saving value
- Number of jobs

Output:

- Selected job IDs
- Final job schedule
- Maximum total energy-saving value

Objective:

To select a set of non-overlapping jobs that maximises the total energy-saving value.

Constraints:

1. Only one job can execute at a time.
2. Overlapping jobs cannot be selected together.
3. Every job has a fixed start and finish time.
4. Each selected job contributes its energy-saving value.
5. A job that finishes at time t and another job that starts at time t are considered non-overlapping.
6. The total value must be maximised.

Decisions to be made:

The algorithm must decide whether to select or reject each job. It must consider the compatibility of jobs and their energy-saving values before making the final schedule.

Given job data

Job	Start Time	Finish Time	Energy-Saving Value
J1	1	3	20
J2	2	5	50
J3	4	6	30
J4	6	8	40
J5	5	9	60
J6	8	10	30
J7	9	11	50
J8	11	13	40

Stage 2 – Propose the Strategy

Possible strategies

There are different strategies that can be used to select jobs.

Strategy	Explanation	Limitation
Earliest finishing time	Selects the job that finishes first.	May not give maximum energy-saving value.
Highest energy-saving value	Selects the job with the highest value first.	May block multiple compatible jobs.
Shortest duration	Selects jobs with smaller execution durations.	May ignore jobs with higher total value.
Dynamic programming	Compares including and excluding jobs using previous results.	Requires storing and calculating states.

Selected strategy: Dynamic Programming

Justification:

Dynamic programming is selected because every job has a different energy-saving value and execution time. Selecting only the highest-value job may prevent other valuable jobs from being selected. Dynamic programming compares both inclusion and exclusion decisions and finds the maximum total value from compatible jobs.

It also stores the results of smaller problems and uses them to calculate the optimal schedule.

Stage 3 – Identify the Critical Information

1. Preprocessing

First, arrange all jobs in ascending order of their finish times.

The algorithm must identify the previous compatible job for every current job.

Let:

[p(i)=text{Latest compatible earlier job} \]

A previous job j is compatible with job i when:

[Finish[j]\leq Start[i] \]

If no compatible job exists, p(i) = 0.

2. Define the Dynamic Programming State

[OPT[i]]

OPT[i] represents the maximum total energy-saving value obtained by considering the first i jobs.

3. Identify the Decisions

There are two decisions for every job.

Include the current job:

If the current job is selected, add its energy-saving value to the best result obtained from its previous compatible jobs.

[Include=Value[i]+OPT[p(i)]]

Exclude the current job:

If the current job is rejected, use the best result obtained from the previous job.

[Exclude=OPT[i-1]]

4. Recurrence Relation

[OPT[i]= max left (OPT[i-1], Value[i]+OPT[p(i)] right)]

5. Base Condition

[OPT[0]=0]

When no jobs are available, the maximum energy-saving value is zero.

Explanation:

The algorithm compares the include and exclude values for every job. It stores the greater value in the DP table. This helps the algorithm find the best combination of non-overlapping jobs.

Stage 4 – Design Complete Pseudocode

Weighted Interval Scheduling Pseudocode

ALGORITHM EnergyAwareScheduling(Jobs, n)

INPUT:

Jobs containing Start, Finish and Value

Number of jobs n

OUTPUT:

Selected Jobs

Maximum Energy-Saving Value

```
BEGIN
  SORT Jobs by ascending Finish Time
  FOR i ← 1 TO n DO
    p[i] ← 0
    FOR j ← i-1 DOWNTO 1 DO
      IF Finish[j] ≤ Start[i] THEN
        p[i] ← j
```

```
            BREAK
        END IF
    END FOR
END FOR
OPT[0] ← 0
FOR i ← 1 TO n DO
    Include ← Value[i] + OPT[p[i]]
    Exclude ← OPT[i-1]
    OPT[i] ← MAX(Include, Exclude)
END FOR
Selected Jobs ← EMPTY
i ← n
WHILE i > 0 DO
    Include ← Value[i] + OPT[p[i]]
    Exclude ← OPT[i-1]
    IF Include > Exclude THEN
        ADD Jobs[i].ID TO SelectedJobs
        i ← p[i]
    ELSE
        i ← i-1
    END IF
END WHILE
REVERSE Selected Jobs
Maximum Value ← OPT[n]
RETURN Selected Jobs, Maximum Value
END
```

Explanation of the Pseudocode

1. The algorithm receives all jobs with their start times, finish times and energy-saving values.
2. It sorts the jobs according to ascending finish time.
3. It identifies the latest compatible previous job for every job.

4. It initialises the dynamic programming table with zero.
5. It calculates the include value and exclude value for every job.
6. It stores the maximum value in the DP table.
7. It traces backwards to identify the selected jobs.
8. It returns the selected jobs and maximum energy-saving value.

Important condition:

\[Finish[j]\leq Start[i] \]

This condition ensures that two jobs finishing and starting at the same time are treated as compatible.

Stage 5 – Desk-Check

Step 1: Arrange the jobs

The given jobs are already arranged according to ascending finish time.

Job	Start	Finish	Value
J1	1	3	20
J2	2	5	50
J3	4	6	30
J4	6	8	40
J5	5	9	60
J6	8	10	30
J7	9	11	50
J8	11	13	40

Step 2: Find the Previous Compatible Jobs

Job	Start Time	Previous Compatible Job	p(i)
J1	1	None	0
J2	2	None	0
J3	4	J1	1
J4	6	J3	3
J5	5	J2	2

J6	8	J4	4
J7	9	J5	5
J8	11	J7	7

Step 3: Calculate the DP Values

Base condition:

$$ OPT[0]=0 $$

For J1:

$$ OPT[1]=\max(0,20)=20 $$

For J2:

$$ OPT[2]=\max(20,50)=50 $$

For J3:

$$ OPT[3]=\max(50,30+20)=50 $$

For J4:

$$ OPT[4]=\max(50,40+50)=90 $$

For J5:

$$ OPT[5]=\max(90,60+50)=110 $$

For J6:

$$ OPT[6]=\max(110,30+90)=120 $$

For J7:

$$ OPT[7]=\max(120,50+110)=160 $$

For J8:

$$ OPT[8]=\max(160,40+160)=200 $$

Step 4: Dynamic Programming Table

Job	Include Value	Exclude Value	OPT[i]
J1	20	0	20
J2	50	20	50
J3	50	50	50
J4	90	50	90

J5	110	90	110
J6	120	110	120
J7	160	120	160
J8	200	160	200

Step 5: Reconstruct the Selected Jobs

Start from J8 and trace backwards.

Job	Include Value	Exclude Value	Decision
J8	200	160	Select J8
J7	160	120	Select J7
J5	110	90	Select J5
J2	50	20	Select J2

Therefore, the selected jobs are:

J2, J5, J7 and J8.

Final Schedule

Selected Job	Start Time	Finish Time	Energy-Saving Value
J2	2	5	50
J5	5	9	60
J7	9	11	50
J8	11	13	40
Total			200

Final Answer:

[50+60+50+40=boxed{200}]

Maximum energy-saving value = 200.

The selected jobs do not overlap because each job finishes at the same time or before the next selected job starts.

Stage 6 – Attack Your Own Design

1. Construct a New Set of Jobs

To test the algorithm, a new set of five jobs is created. The purpose is to check whether selecting a job with the highest individual value can lead to a poor overall result.

Adversarial dataset

Job	Start Time	Finish Time	Energy-Saving Value
A	1	9	100
B	1	4	45
C	4	7	45
D	7	10	45
E	10	12	20

2. Explain Why the Input Is Challenging

Job A has the highest individual energy-saving value of 100. However, selecting job A prevents jobs B, C and D from being selected together.

Jobs B, C and D are compatible with one another.

Their combined value is:

\[45+45+45=135 \]

This is greater than the value of job A.

Therefore, the input challenges a simple highest-value-first strategy.

3. Prediction Before Applying the Algorithm

The expected result is that the algorithm will select jobs B, C, D and E because their combined energy-saving value is greater than selecting A and E.

4. Apply the Algorithm

The jobs are arranged according to ascending finish time.

Job	Finish Time	Previous Compatible Job	p(i)
B	4	None	0
C	7	B	1
A	9	None	0
D	10	C	2
E	12	D	4

Base condition:

$$ OPT[0]=0 $$

For B:

$$ OPT[1]=\max(0,45)=45 $$

For C:

$$ OPT[2]=\max(45,45+45)=90 $$

For A:

$$ OPT[3]=\max(90,100)=100 $$

For D:

$$ OPT[4]=\max(100,45+90)=135 $$

For E:

$$ OPT[5]=\max(135,20+135)=155 $$

5. Adversarial DP Table

Job	Include Value	Exclude Value	OPT[i]
B	45	0	45
C	90	45	90
A	100	90	100
D	135	100	135
E	155	135	155

6. Final Selected Jobs

The algorithm selects:

- Job B = 45
- Job C = 45
- Job D = 45
- Job E = 20

Total energy-saving value:

$$ 45+45+45+20=\boxed{155} $$

Comparison of the two possible schedules

Schedule	Selected Jobs	Total Value
Schedule 1	A, E	120

Schedule 2 B, C, D, E 155

The second schedule gives the maximum energy-saving value.

7. Conclusion

The prediction is correct. The dynamic programming algorithm successfully selects jobs B, C, D and E instead of selecting only the highest-value job A.

The critical decision is comparing the include and exclude values for each job. The algorithm considers the best compatible previous schedule before selecting the current job.

This example demonstrates that a locally attractive decision may not produce a globally optimal result. Dynamic programming solves the problem by considering combinations of compatible jobs.

OVERALL CONCLUSION

The assignment demonstrates the importance of algorithmic design using greedy strategies and dynamic programming.

Task 1 explains the Fractional Knapsack problem and shows how a greedy algorithm selects packages according to value per unit weight.

Task 2 demonstrates Job Sequencing with Deadlines using descending profit order and available time slots.

Task 3 explains the 0/1 Knapsack problem using dynamic programming states, recurrence relations and item reconstruction.

Task 4 identifies and repairs a logical defect in the Word Break algorithm using a counterexample and regression testing.

Task 5 compares Dijkstra, Bellman-Ford and Floyd's algorithms for different shortest-path requirements.

Task 6 develops an Energy-Aware Job Scheduling algorithm using weighted interval scheduling and dynamic programming. It also tests the algorithm using an adversarial dataset.

Through these tasks, the assignment develops problem-solving, algorithm design, pseudocode writing, manual tracing, debugging and optimization skills.

REFERENCES

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, MIT Press.
2. Jon Kleinberg and Éva Tardos, Algorithm Design, Pearson.
3. Ellis Horowitz, Sartaj Sahni and Sanguthevar Rajasekaran, Fundamentals of Computer Algorithms.
4. SIMATS Engineering, Handbook for Algorithmic Pseudocode